

OptiCrop – Smart Agricultural Production Optimization Engine

Project Description:

The project "Smart Agricultural Production Optimization Engine" aims to develop an advanced software system that utilizes data-driven insights to optimize agricultural production for different crops. By integrating key environmental factors such as Nitrogen (N), Phosphorous (P), Potassium (K) levels, soil temperature, humidity, pH, rainfall, and crop types, OptiCrop seeks to provide intelligent recommendations to farmers for maximizing yields and resource efficiency. The knowledge and insights gained through the OptiCrop project can be utilized by agricultural researchers, policymakers, and stakeholders to advance the understanding of crop-environment interactions and inform the development of evidence-based agricultural strategies. The OptiCrop project revolutionizes agricultural practices by providing farmers with a smart production optimization engine. By integrating key environmental factors and crop-specific recommendations, OptiCrop enables farmers to achieve optimal yields, improve resource efficiency, and contribute to sustainable agricultural practices.

Scenarios

Scenario 1: Smart Crop Recommendation for Farmers

A farmer enters soil and environmental details such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall. The system analyzes the data and recommends the most suitable crop for maximum yield and better farming efficiency.

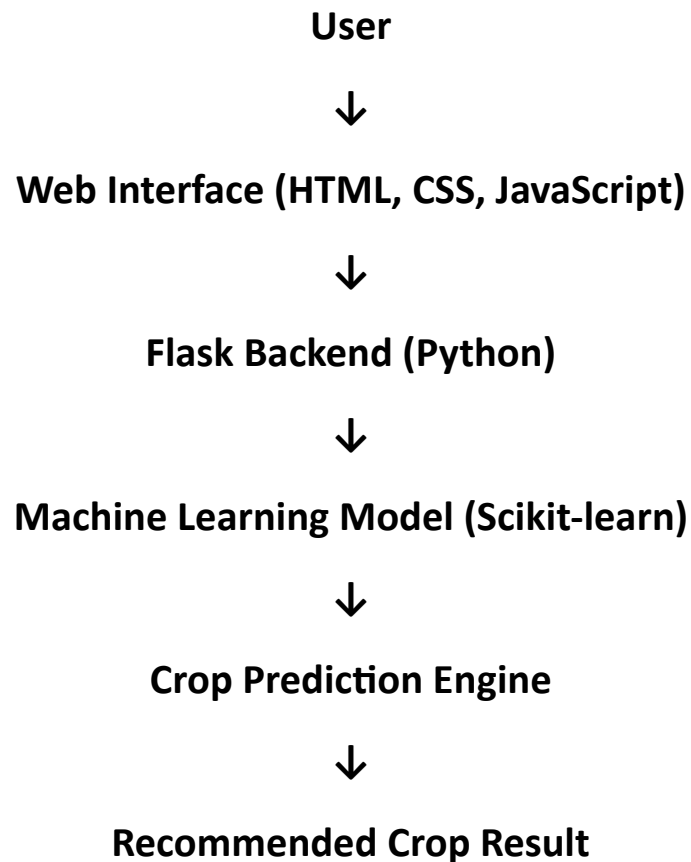
Scenario 2: Crop Suitability and Environmental Assessment

A user wants to understand whether the current soil and climate conditions are suitable for a particular crop. The application evaluates the input parameters and provides insights about crop compatibility, environmental suitability, and productivity potential.

Scenario 3: Agricultural Research and Policy Planning

An agricultural researcher or policymaker uses the system to analyze crop-environment relationships and identify patterns in agricultural production. The platform helps in making data-driven decisions for sustainable farming strategies and resource optimization.

Technical Architecture:



Description:

The OptiCrop system follows a three-layer architecture consisting of the User Interface Layer, Application Layer, and Machine Learning Layer. The frontend is developed using HTML, CSS, and JavaScript, allowing users to enter agricultural parameters such as Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall. The Flask backend processes the user input, validates the data, and sends it to the trained Machine Learning model.

The Machine Learning model, developed using Python and Scikit-learn, analyzes the input parameters and predicts the most suitable crop for cultivation. The prediction result is then returned to the Flask application and displayed to the user through the web interface. This architecture provides an efficient, user-friendly, and intelligent crop recommendation system that assists farmers in making data-driven agricultural decisions and improving crop productivity.

Pre-requisites:

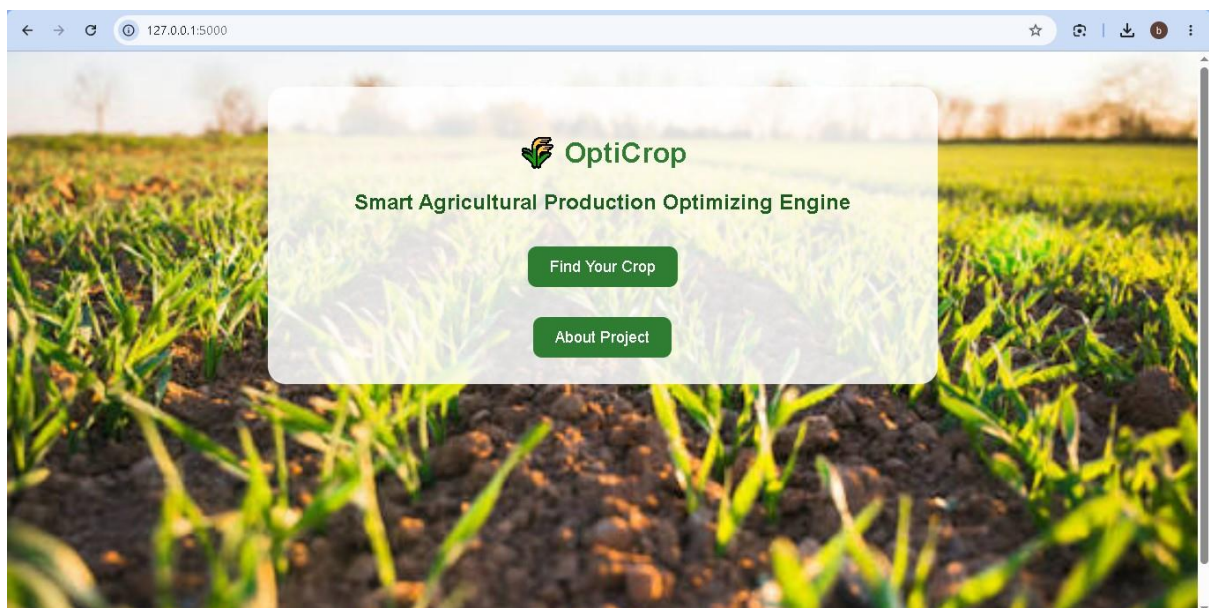
- Python
- Flask
- Pandas
- NumPy
- Scikit-learn
- HTML
- CSS
- JavaScript
- Git and GitHub

Project Workflow:

Epic 1: Define Problem and Understanding

Story 1: Identify and define the agricultural problem that the crop recommendation system aims to solve.

Story 2: Gather and analyze business requirements to understand project objectives and expected outcomes.



Story 3: Conduct a literature survey to study existing crop recommendation techniques and machine learning approaches.

Story 4: Analyze the social and business impact of implementing an AI-driven crop recommendation solution.



Epic 2: Data Collection and Analysis

Story 1: Download and collect the agricultural dataset from reliable sources.

Story 2: Import the required Python libraries for data analysis, visualization, and machine learning.

Story 3: Read and explore the dataset to understand its structure, features, and target variables.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
1	N	P	K	temperature	humidity	ph	rainfall	label													
2	90	42	43	20.87974	82.00274	6.502985	202.9355	rice													
3	85	58	41	21.77046	80.31964	7.038096	226.6555	rice													
4	60	55	44	23.00446	82.32076	7.940207	263.9642	rice													
5	74	35	40	26.4911	80.15836	6.980401	242.864	rice													
6	78	42	42	20.13017	81.60487	7.628473	262.7173	rice													
7	69	37	42	23.05805	83.37012	7.073454	251.055	rice													
8	69	55	38	22.70884	82.63941	5.700806	271.3249	rice													
9	94	53	40	20.27774	82.89409	5.718627	241.9742	rice													
10	89	54	38	24.51588	83.53522	6.685346	230.4462	rice													
11	68	58	38	23.22397	83.03323	6.336254	221.2092	rice													
12	91	53	40	26.52724	81.41754	5.386168	264.6149	rice													
13	90	46	42	23.97898	81.45062	7.502834	250.0832	rice													
14	78	58	44	26.8008	80.98689	5.108682	284.4365	rice													
15	93	56	36	24.01498	82.05687	6.984354	185.2773	rice													
16	94	50	37	25.66585	80.66385	6.94802	205.587	rice													
17	60	48	39	24.28209	80.30026	7.042299	231.0863	rice													
18	85	38	41	21.58712	82.78837	6.249051	276.6552	rice													
19	91	35	39	23.79392	80.41818	6.97086	206.2612	rice													
20	77	38	36	21.86525	80.1923	5.953933	224.555	rice													
21	88	35	40	23.57944	83.5876	5.853932	291.2987	rice													
22	89	45	36	21.32504	80.47476	6.442475	185.4975	rice													

Story 4: Perform univariate analysis to examine the distribution and characteristics of individual features.

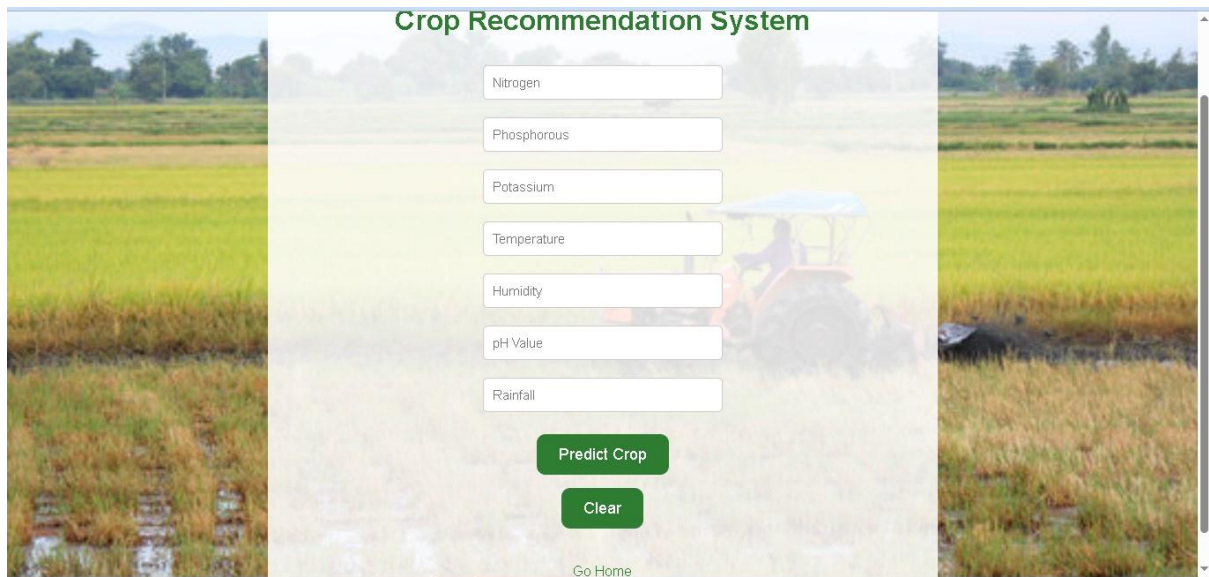
Story 5: Conduct bivariate analysis to identify relationships between agricultural parameters and crop suitability.

Story 6: Perform multivariate analysis to discover patterns and interactions among multiple variables.

Epic 3: Data Pre-Processing

Story 1: Check for null values and handle missing data to improve dataset quality.

Story 2: Detect and treat outliers that may affect model performance.



The screenshot shows a web application titled "Crop Recommendation System". It features a central form with seven input fields for agricultural data: Nitrogen, Phosphorous, Potassium, Temperature, Humidity, pH Value, and Rainfall. Below these fields are two green buttons: "Predict Crop" and "Clear". At the bottom of the form is a "Go Home" link. The background of the page is a blurred image of a green field with a tractor.

Crop Recommendation System

Nitrogen

Phosphorous

Potassium

Temperature

Humidity

pH Value

Rainfall

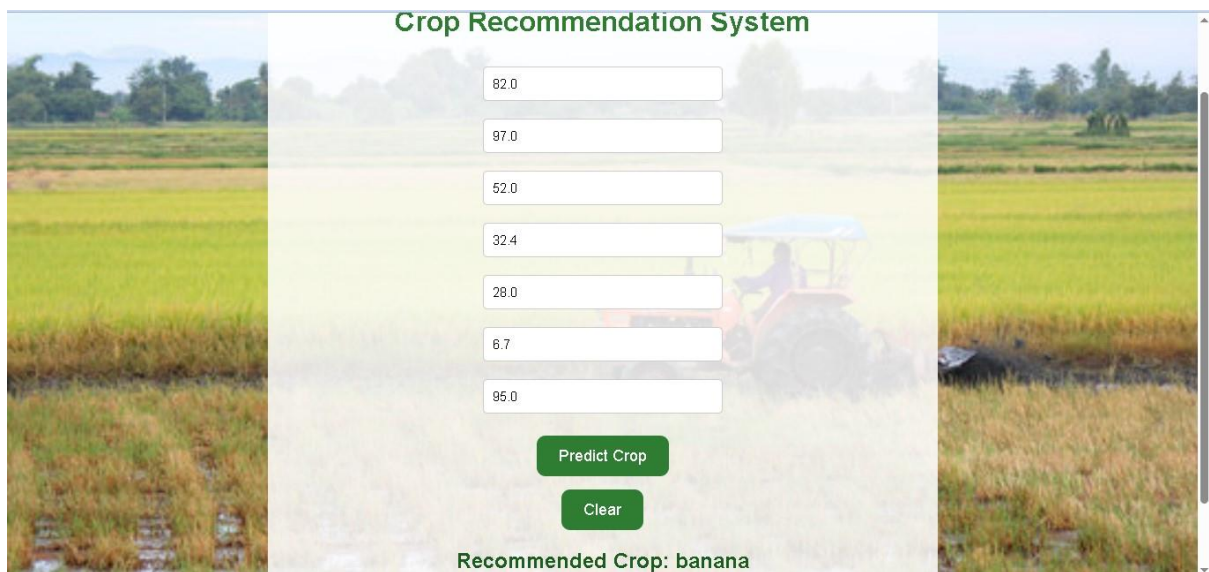
Predict Crop

Clear

Go Home

Story 3: Extract seasonal crop information and prepare relevant features for analysis.

Story 4: Split the dataset into training and testing sets for model development and evaluation.



This screenshot shows the same web application as the previous one, but with numerical values entered into the input fields. The "Predict Crop" button has been clicked, and the result "Recommended Crop: banana" is displayed at the bottom of the form. The "Clear" button remains visible.

Crop Recommendation System

82.0

97.0

52.0

32.4

28.0

6.7

95.0

Predict Crop

Clear

Recommended Crop: banana

Epic 4: Model Building

Story 1: Apply K-Means Clustering to identify patterns and group similar agricultural conditions.

Story 2: Train a Logistic Regression model to predict suitable crops based on environmental parameters.

```
1 import pandas as pd
2 import joblib
3
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score
6
7 from sklearn.ensemble import RandomForestClassifier
8 from sklearn.tree import DecisionTreeClassifier
9 from sklearn.neighbors import KNeighborsClassifier
10 from sklearn.linear_model import LogisticRegression
11
12 # Load dataset
13 df = pd.read_csv("Crop_recommendation.csv")
14
15 # Features and target
16 X = df.drop("label", axis=1)
17 y = df["label"]
18
19 # Split dataset into training and testing data
20 X_train, X_test, y_train, y_test = train_test_split(
21     X, y, test_size=0.2, random_state=42
22 )
23
24 # Create models
25 models = {
26     "Random Forest": RandomForestClassifier(
27         n_estimators=100, random_state=42
28     ),
29     "Decision Tree": DecisionTreeClassifier(
30         random_state=42
31     ),
32     "KNN": KNeighborsClassifier(
```

Story 3: Evaluate model performance using appropriate metrics and compare results.

Story 4: Save the best-performing model for deployment and generate crop predictions based on user-provided inputs.

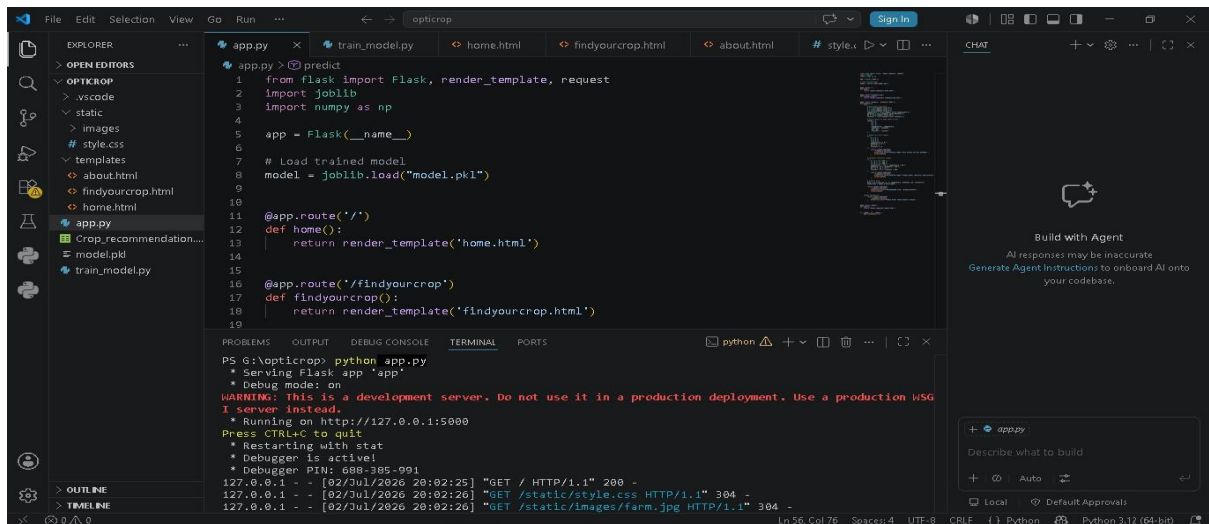
Epic 5: Application Building

Story 1: Design and develop HTML pages to create an interactive and user-friendly interface.

Story 2: Build the Python backend code and integrate it with the trained machine learning model.

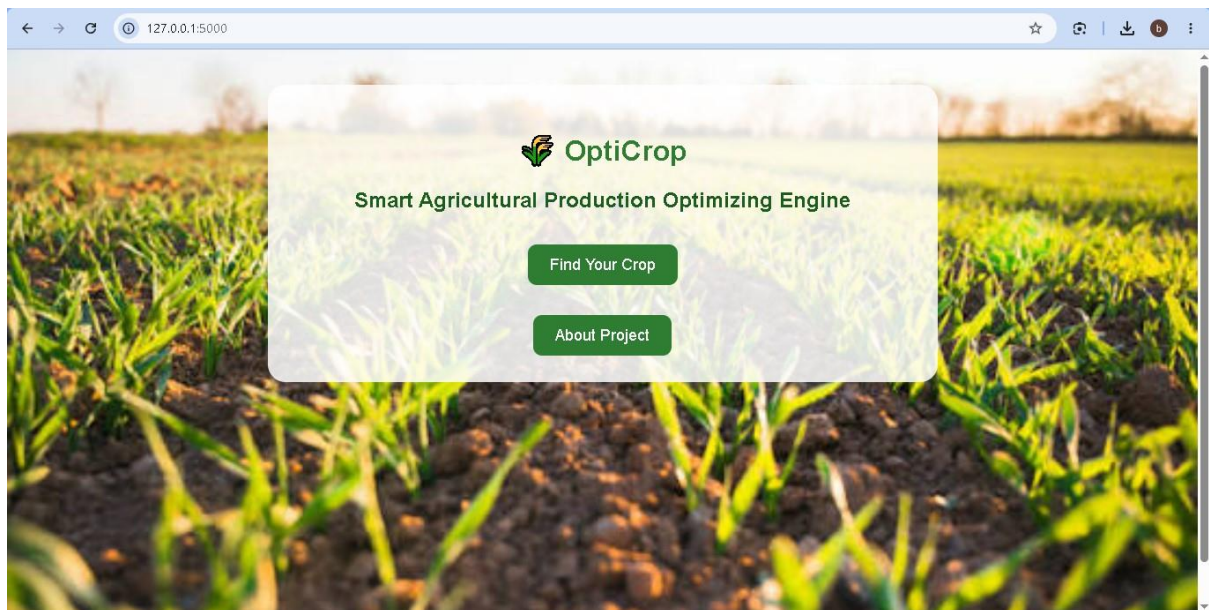
Story 3: Run, test, and validate the application to ensure accurate crop recommendations and smooth functionality.

```
1 from flask import Flask, render_template, request
2 import joblib
3 import numpy as np
4
5 app = Flask(__name__)
6
7 # Load trained model
8 model = joblib.load("model.pkl")
9
10
11 @app.route('/')
12 def home():
13     return render_template("home.html")
14
15
16 @app.route('/findyourcrop')
17 def findyourcrop():
18     return render_template("findyourcrop.html")
19
20
21 @app.route('/predict', methods=['POST'])
22 def predict():
23     try:
24         # Get values from form
25         N = float(request.form['N'])
26         P = float(request.form['P'])
27         K = float(request.form['K'])
28         temperature = float(request.form['temperature'])
29         humidity = float(request.form['humidity'])
30         ph = float(request.form['ph'])
31         rainfall = float(request.form['rainfall'])
```



Exploring the Website's Web Pages:

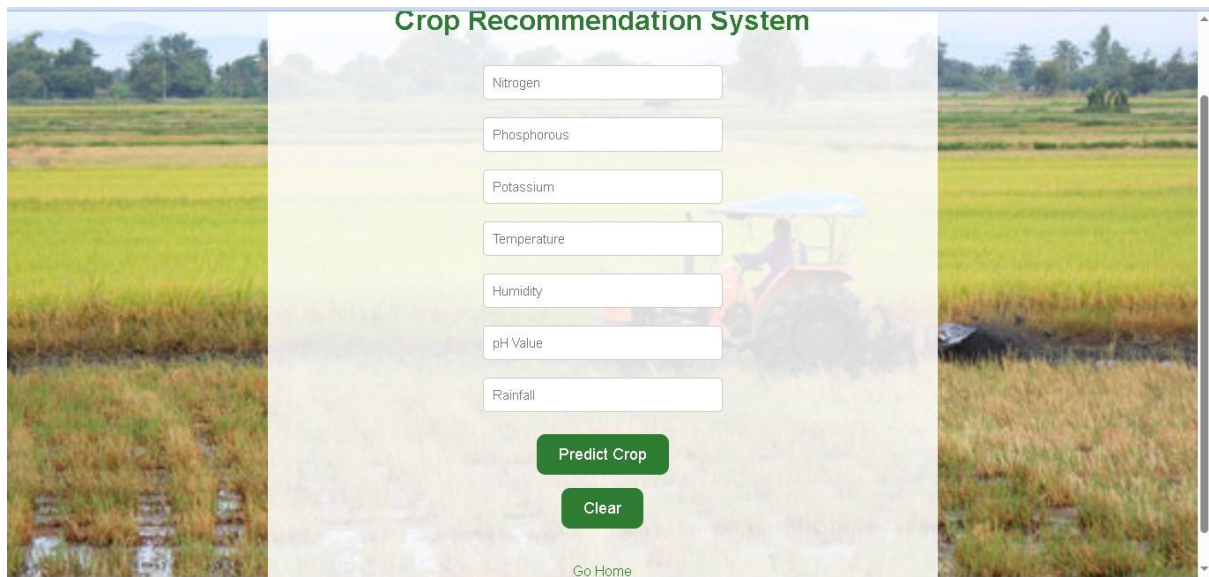
Home Page:



Description:

The Home Page is the main entry point of the OptiCrop – Smart Agricultural Production Optimization Engine. It provides an attractive and user-friendly interface with an agricultural background image and introduces the project to users. The page contains two main buttons: Find Your Crop and About Project. The "Find Your Crop" button redirects users to the crop prediction page, while the "About Project" button provides information about the project and its objectives. The home page is designed to give users easy navigation and a professional look.

Input Section:

The screenshot shows a web application titled "Crop Recommendation System" in green text. The background is a blurred image of a green field with a tractor. The form is centered and contains seven white input fields with green borders, stacked vertically. The fields are labeled: "Nitrogen", "Phosphorous", "Potassium", "Temperature", "Humidity", "pH Value", and "Rainfall". Below these fields are two green buttons: "Predict Crop" and "Clear". At the bottom center, there is a small green link that says "Go Home".

Crop Recommendation System

Nitrogen

Phosphorous

Potassium

Temperature

Humidity

pH Value

Rainfall

Predict Crop

Clear

[Go Home](#)

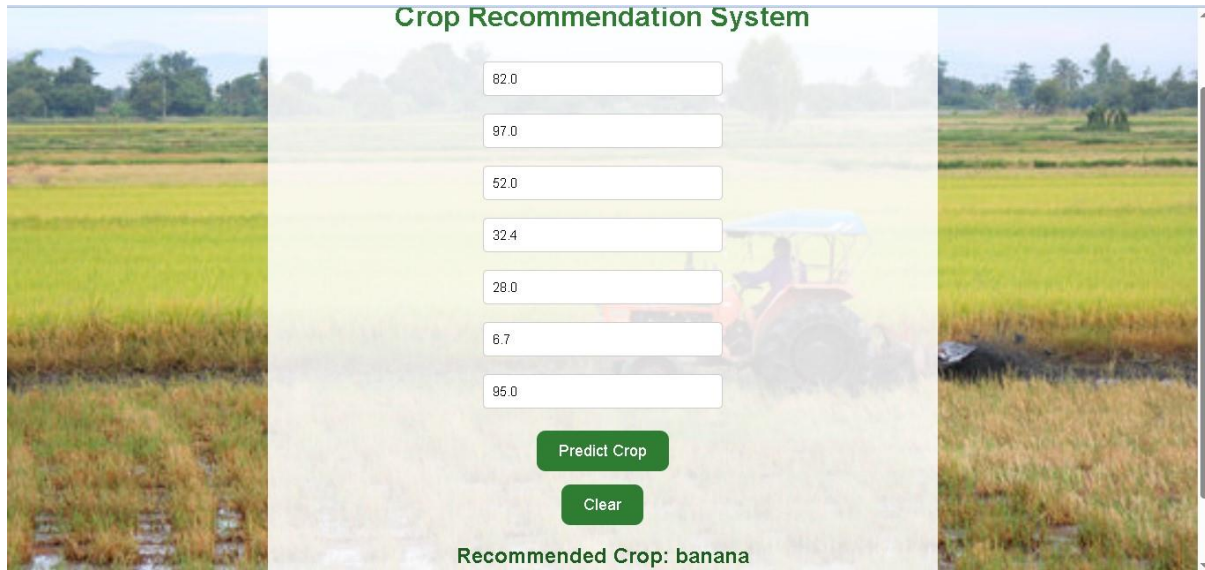
Description:

The Find Your Crop page is the core functionality of the project. This page allows users to enter important agricultural parameters such as:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH Value
- Rainfall

After entering these values, the user clicks the Predict Crop button. The Flask backend sends the data to the trained Machine Learning model, which analyzes the input and predicts the most suitable crop for cultivation. The predicted crop is then displayed to the user. A Clear button is also provided to reset all input fields.

Result Section:



The screenshot shows a web application titled "Crop Recommendation System". It features a central form with seven input fields containing the following values: 82.0, 97.0, 52.0, 32.4, 28.0, 6.7, and 95.0. Below the inputs are two green buttons: "Predict Crop" and "Clear". At the bottom, it displays "Recommended Crop: banana". The interface is flanked by two vertical images of agricultural fields.

Description:

The OptiCrop – Smart Agricultural Production Optimization Engine was successfully developed and tested. The system accepts agricultural input parameters such as Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall and predicts the most suitable crop using a trained Machine Learning model.

The web application provides accurate and quick crop recommendations through a simple and user-friendly interface. The prediction results help farmers make informed agricultural decisions, improve crop productivity, and select crops that are suitable for their soil and environmental conditions.

The project successfully demonstrates the practical application of Machine Learning in Agriculture by integrating a Flask web application with a crop prediction model.

About Page:



The screenshot shows an "About Project" page with a background image of rice stalks. The text describes the project's goal: to recommend the most suitable crop based on soil nutrients and environmental conditions. It lists the input parameters: Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH Value, and Rainfall. A "Back to Home" link is also present.

About Project

This project uses Machine Learning to recommend the most suitable crop based on soil nutrients and environmental conditions such as Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH and Rainfall. It helps farmers make better agricultural decisions and improve crop productivity.

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH Value
- Rainfall

[Back to Home](#)

Description:

The About Project page provides detailed information about the purpose and working of the AI-ML OptiCrop system. It explains that the project uses Machine Learning techniques to recommend the most suitable crop based on soil nutrients and environmental conditions. The page describes the input parameters used by the system, including Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall. It also highlights the benefits of the system, such as helping farmers make better agricultural decisions and improving crop productivity.

Conclusion:

OptiCrop is an advanced Machine Learning-powered smart agricultural recommendation system designed to support modern, data-driven farming practices by providing accurate and intelligent crop prediction solutions. The system analyzes multiple critical environmental and soil-related parameters including nitrogen, phosphorus, and potassium (NPK) soil nutrient levels, temperature, humidity, soil pH, and rainfall patterns to determine the most suitable crop for cultivation under specific conditions. By combining agricultural science with artificial intelligence, OptiCrop aims to reduce uncertainty in farming decisions and promote efficient, sustainable agricultural production.

The project leverages multiple Machine Learning algorithms such as K-Nearest Neighbors (KNN), K-Means Clustering, and Logistic Regression to perform crop classification and recommendation tasks. Each algorithm contributes uniquely to the prediction process: KNN identifies similar environmental conditions based on historical agricultural data, K-Means helps in grouping and analyzing soil and climate patterns, while Logistic Regression provides accurate classification of the most appropriate crop category. The use of multiple models allows comparative analysis and improves the reliability and robustness of predictions, ensuring better recommendation accuracy across varying agricultural conditions.